

FORECASTING LLM TOKEN DEMAND WITH CONVERSATION STATE

Time-Series Data Analysis Proposal

Team 4 / Lee Junsang · Seo Dael · Roh Jonghyeon · Lim Jihun

BACKGROUND & MOTIVATION

- LLM demand is growing fast
- Too few servers → delays and failures; too many → wasted power → capacity must follow demand
- New servers take minutes to start → reacting is too late; we need to know minutes ahead
- Load depends on input/output length, not just request count → forecast input and output tokens

→ So, can we know input and output token demand in advance?

RELATED WORK

Common gap: none of the four forecasts next demand from the state of ongoing conversations

Prior work · Method & contribution	Limitation
BurstGPT · KDD 2025 — released real request logs; forecasts request and token counts from past aggregates (XGBoost)	Aggregate history only; does not test whether adding conversation state helps
DynamoLLM · HPCA 2025 — tunes server count and GPU settings to request length and load; saves 53% energy while meeting SLOs	Predicts lengths of requests that have arrived; not the next demand of ongoing conversations
PreServe · ICSE 2026 — hierarchically forecasts total token demand and per-request length; feeds proactive allocation and routing	Forecasts total demand, but does not use conversation links
SageServe · Microsoft 2025 — ARIMA demand forecasts; scales servers so model redeployment (~10 min) starts ahead of time	Hourly aggregates only; no 10-minute changes or input/output split

Research question:

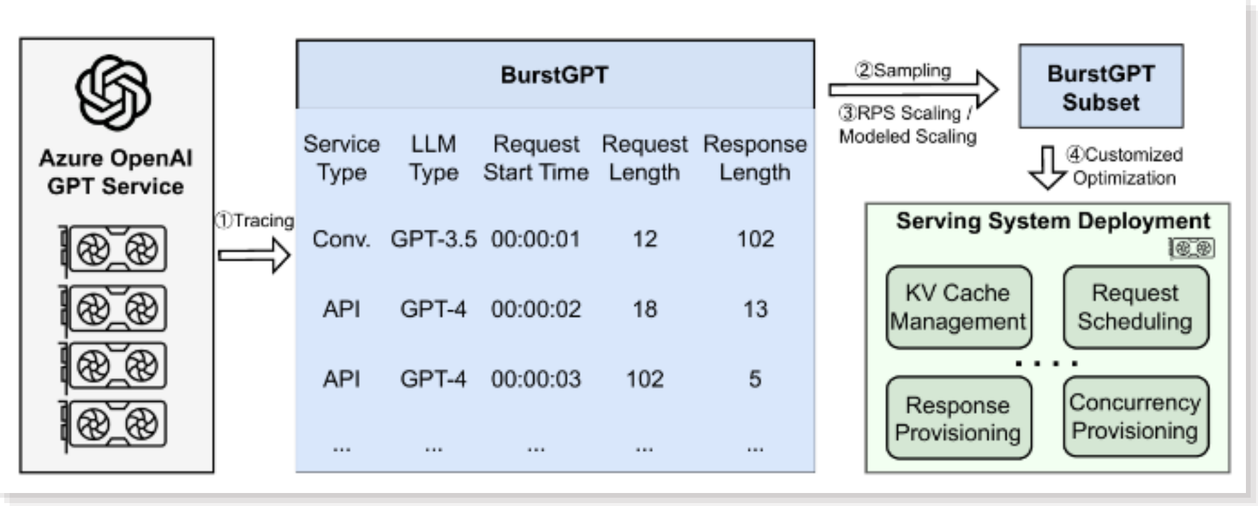
Does adding ongoing conversations to demand history improve 10-minute token forecasts?

OUR HYPOTHESIS

Does adding conversation information improve token demand forecasts? — A → B → C, same origins, same model

Features	Model A	Model B	Model C
Forecasting setup	Time-series ML regression · separate input/output models · same tuning budget		
① Past demand history Lags and rolling stats of request count, input tokens, and completed output tokens; request-length features; time of day at the origin	BASELINE	•	•
② Active conversation count Unique conversations that sent a request in the last 5 · 15 · 30 min		NEW	•
③ Conversation state summary Conversations active in the last 30 min: time since last request, request count, age, last input length and its change			NEW
Hypotheses tested	BASELINE MODEL BASED ON RELATED WORK	H1 · B vs A Count → less error	H2 · C vs B State → less error

DATASET: BURSTGPT_3



- Real GPT service request logs via Azure OpenAI (v2.0)
- 110 days · 5,344,021 requests, 233,617 conversational
- 55,920 conversations · 36,086 with 2+ requests
- Request time · conversation ID · in/out tokens · response time

Source: Wang et al., BurstGPT, KDD 2025 · v2.0 release

HOW THE LOG WAS COLLECTED

- Logging engine installed in a regional GPT service used by 3,000+ people (Azure OpenAI Service)
- Records both conversation and API requests
- Logs submission time, input/output tokens, and model/service type during real use

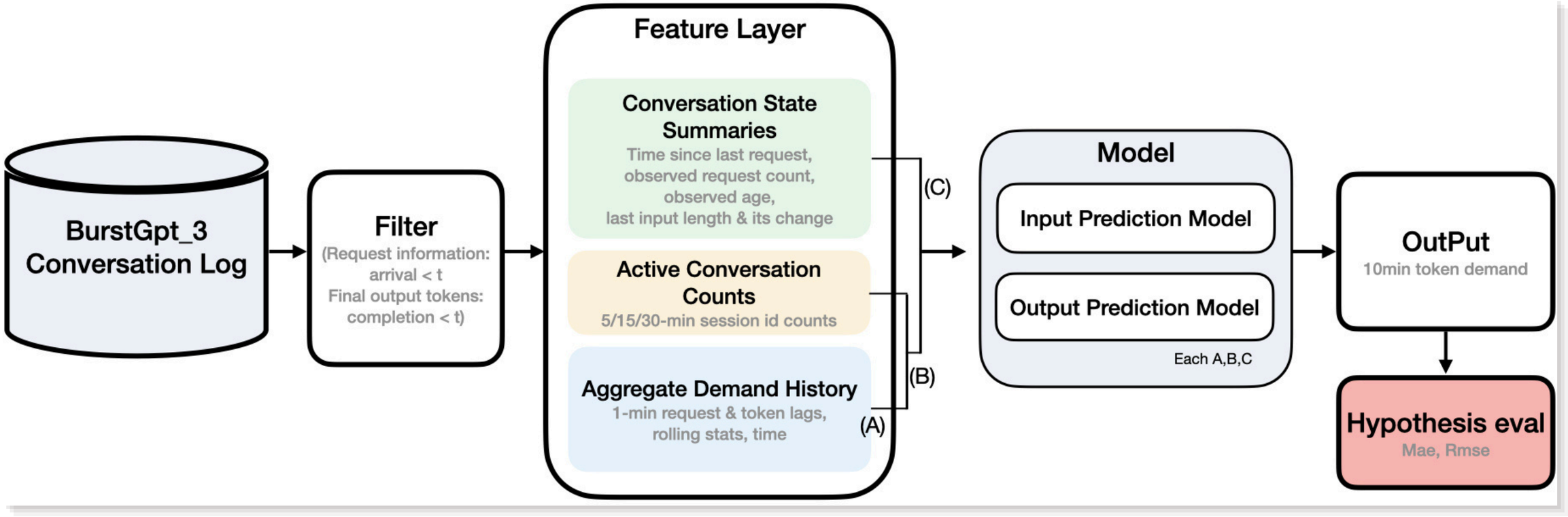
WHY IT FITS to Time series analysis

Timestamps, token counts, and conversation IDs let us model demand over time and test whether conversation activity helps short-term forecasts.

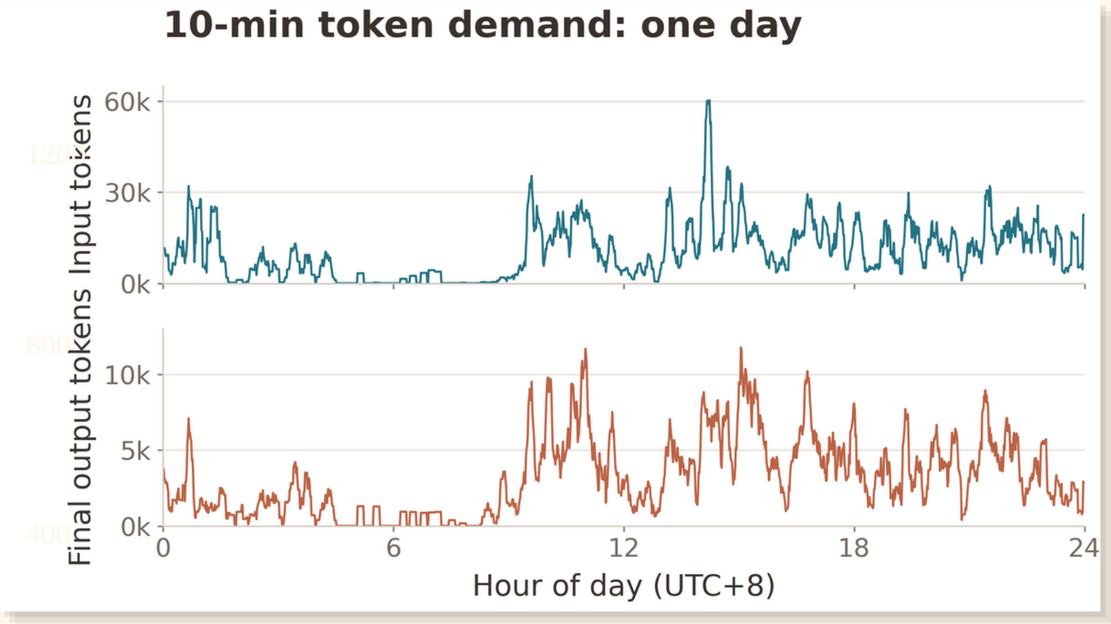
DATA FIELDS

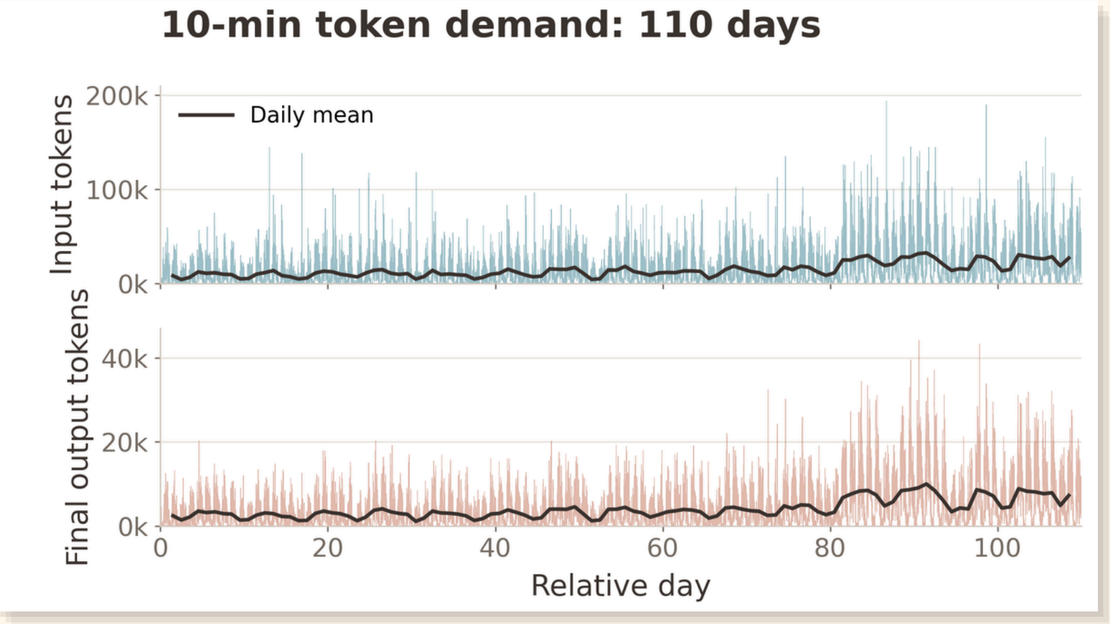
Column	Description
Timestamp	Seconds from reference-day midnight to submission (real date/weekday unknown, UTC+8)
Session ID	Links requests in the same conversation (not a user ID) → key for B·C features
Request / Response / Total tokens	Input length / final output length / sum of both
Elapsed time	Seconds from submission to full response (not TTFT) → completion = submit + Elapsed
Model	ChatGPT or GPT-4
Log Type	Conversation log or API log → this study uses conversation logs only

PIPELINE OVERVIEW



EDA ① TARGET DEMAND





Daily data - short demand peaks appear alongside low-demand periods.
Entire trend - the scale of demand also shifts across the observation period.

(To reflect these patterns)
→ Our prediction: use recent demand history and time of day as common inputs.

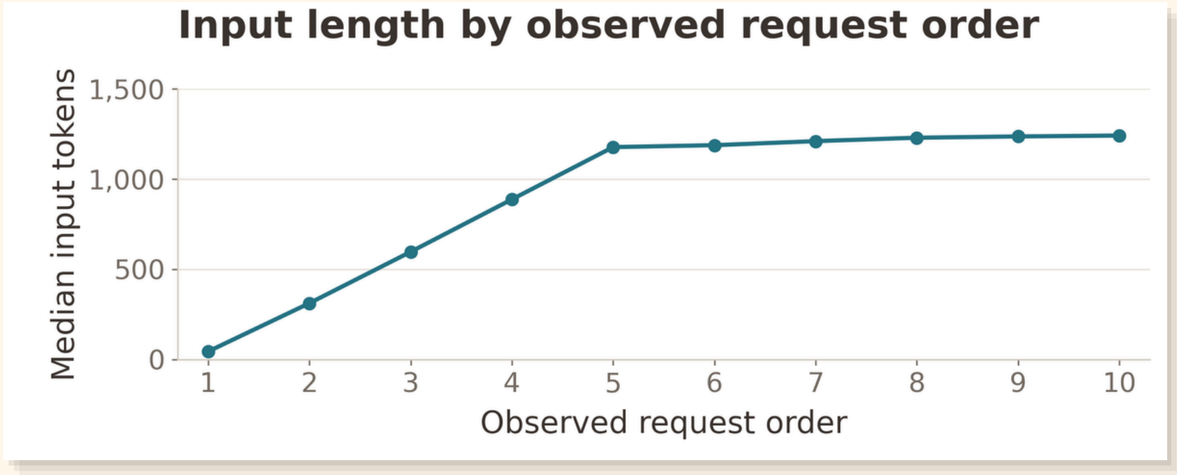
EDA ② CONVERSATIONS

Repeated Conversations

Conversation evidence	Value
Unique conversation IDs	55,920
Conversations with 2+ requests	**64.53%**
Share of requests in repeated conversations	**91.51%**
Observed follow-up gap, median	**131 sec**

Horizon comparison

Horizon	Follow-up gaps ≤ horizon (%)	Zero-demand windows (%)
5 min	66.1%	22.3%
10 min	**76.7%**	**15.8%**
15 min	81.6%	12.6%



Based on this observation

Requests repeat in 64.5% of conversations, and median input length varies with request order → recent conversation activity and state are candidate inputs.

Follow-up gap coverage rises 10.6 pp from 5→10 min but only 4.9 pp from 10→15 min → we set a 10-minute horizon.

EDA ③ CORRELATIONS

EXPLORATORY SPEARMAN CORRELATIONS

Past predictor	Next 10m input	Next 10m output
Requests (last 5 min)	0.822	0.832
Active conversations (last 5 min)	**0.825**	**0.841**

Past predictor

Recent 5-min request count and active conversations both correlate strongly with input/output tokens → both are candidate predictors

However, **Past request count vs. active conversations (5 min) Spearman correlation = 0.978**

Based on this Result

Same target, chronological split, and model; we add different inputs and compare input/output forecast errors separately.

B vs A shows the added effect of active conversation count; C vs B shows the added effect of conversation state.

WORKLOAD DISTRIBUTION

Plan Data QC → Features & labels → Split & baseline → A / B / C comparison → Interpret & report

Lee Junsang	Model development — input/output XGBoost, chronological validation, equal tuning budget
Seo Dael	Feature engineering & targets — 10-min labels, aggregate/session features, no-leakage availability
Roh Jonghyeon	Preprocessing & EDA — conversation-log prep, quality checks, temporal patterns of demand
Lim Jihun	Evaluation & interpretation — error by feature set and period, added value of conversation state
Shared	Experiment design, leakage checks, reproducibility, final report and presentation

BEYOND THIS PROJECT

- This project: does conversation state reduce next-10-min input/output token forecast error?
- Out of scope: using forecasts for server scaling, latency, actual power use
- Tuning servers/GPU settings to load saves 53% energy (DynamoLLM, HPCA 2025)
- To do that proactively, demand must be known in advance → forecastability comes first

→ Future work: if forecasts are good, does proactive scaling cut power and latency?



Questions?

Thank you

Team 4 / Lee Junsang · Seo Dael · Roh Jonghyeon · Lim Jihun